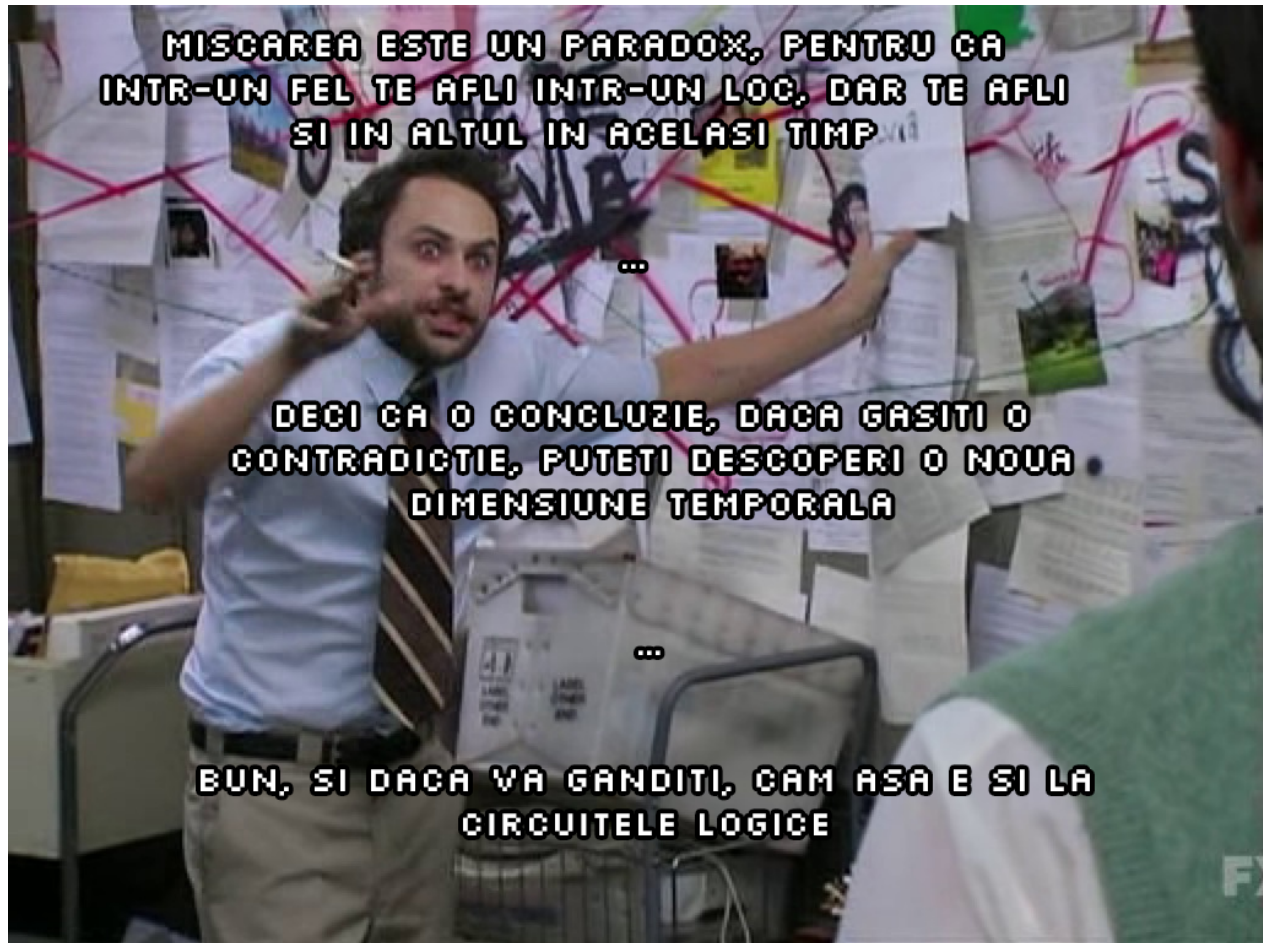


Tutoriat 4

Stan Bianca-Mihaela, Stăncioiu Silviu

November 23, 2020



Contents

1	Algebra Booleana	2
1.1	Operatii si proprietati	2
1.2	Functiile booleene	4
2	Porti logice	5
3	MIPS	12
3.1	Șiruri de caractere	12
3.2	Lucrul cu numere în virgulă mobilă	15

1 Algebra Booleana

1.1 Operatii si proprietati

Vom incepe cu algebra booleana cu care probabil sunteti deja familiari de la Logica.
Ca un mic twist, la ASC avem notatiile:

- $+ \Leftrightarrow \vee$ (SAU)
- $\cdot \Leftrightarrow \wedge$ (SI)
- $\overline{} \Leftrightarrow \neg$ (NEGATIA)

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

$$\overline{0} = 1$$

$$\overline{1} = 0$$

Dupa cum stim, pe o algebra booleana B_2 (aka unde avem doar multimea de valori 0,1) avem anumite axiome:

1. ASOCIATIVITATEA

$$(x+y)+z=x+(y+z)$$

$$(x \cdot y) \cdot z = x \cdot (y \cdot z)$$

2. COMUTATIVITATEA

$$x+y=y+x$$

$$x \cdot y = y \cdot x$$

3. ABSORBTIA

$$x + (x \cdot y) = x \cdot (1 + y) = x \text{ (fiindca } 1 + y = 1 \text{ sau "ceva" va da mereu 1)}$$

$$x \cdot (x + y) = x \text{ (daca } x \text{ e } 0 \Rightarrow \text{rezultatul e } 0, \text{ daca } x \text{ e } 1 \Rightarrow \text{rezultatul e } 1 \text{ indiferent de } y)$$

4. DISTRIBUTIVITATEA

$$x \cdot (y + z) = x \cdot y + x \cdot z$$

$$x + (y \cdot z) = (x + y) \cdot (x + z)$$

5. MARGINIREA

$$0+x=x$$

$$0 \cdot x = 0$$

$$1+x=1$$

$$1 \cdot x = x$$

6. COMPLEMENTAREA

$$x + \bar{x} = 1$$

$$x \cdot \bar{x} = 0$$

Pe langa acestea, mai definim si cateva operatii derivate:

- IMPLICATIA

$$x \rightarrow y = \bar{x} + y$$

- DIFERENTA

$$x - y = x \cdot \bar{y}$$

- ECHIVALENTA

$$x \leftrightarrow y = (x \rightarrow y) \cdot (y \rightarrow x) = (\bar{x} + y) \cdot (\bar{y} + x)$$

- XOR

$$x \oplus y = x \cdot \bar{y} + \bar{x} \cdot y$$

- NXOR

$$x * y = \overline{x \oplus y} \text{ (am notat cu } * \text{ pentru ca NXOR nu are un semn propriu-zis din cate stiu eu)}$$

- NAND

$$x * y = \overline{x \cdot y} \text{ (am notat cu } * \text{ pentru ca NAND nu are un semn propriu-zis din cate stiu eu)}$$

- NOR

$$x * y = \overline{x + y} \text{ (am notat cu } * \text{ pentru ca NOR nu are un semn propriu-zis din cate stiu eu)}$$

Aici avem un tabel de adevar cu cele mai importante operatii:

x	y	$x \cdot y$	x NAND y	$x + y$	x NOR y	$x - y$	$x \rightarrow y$	$x \oplus y$	x NXOR y
0	0	0	1	0	1	0	1	0	1
0	1	0	1	1	0	0	1	1	0
1	0	0	1	1	0	1	0	1	0
1	1	1	0	1	0	0	1	0	1

Si cateva proprietati:

1. IDEMPOTENTA

$$x+x=x$$

$$x \cdot x = x$$

2. LEGEA DUBLEI NEGATII

$$\bar{\bar{x}} = x$$

3. LEGILE LUI MORGAN

$$\overline{x + y} = \bar{x} \cdot \bar{y}$$

$$\overline{x \cdot y} = \bar{x} + \bar{y}$$

$$\overline{\bar{x} + \bar{y}} = x \cdot y$$

$$\overline{\bar{x} \cdot \bar{y}} = x + y$$

4. ABSORBTIA BOOLEANA

$$x + \bar{x} \cdot y = x + y$$

$$x \cdot (\bar{x} + y) = x \cdot y$$

5. UNICITATEA COMPLEMENTULUI

$$x+y=1 \text{ si } x \cdot y = 0 \Rightarrow y = \bar{x}$$

$$x+y=0 \Rightarrow x = y = 0$$

$$x \cdot y = 1 \Rightarrow x = y = 1$$

1.2 Functiile booleene

O functie booleana este o functie de forma:

$$f : B_2^n \rightarrow B_2^k \Leftrightarrow f : \{0, 1\}^n \rightarrow \{0, 1\}^k$$

Pentru aceste functii putem defini FND si FNC cu care probabil cuneti familiari de la logica.

- FND (FUNCTIA NORMALA DISJUNCTIVA) aka unde da functia 1
 $(\cdot) + (\cdot) + \dots + (\cdot) = 1$
- FNC (FUNCTIA NORMALA CONJUNCTIVA) aka unde da functia 0
 $(+) \cdot (+) \cdot \dots \cdot (+) = 0$

Exemplul 1 : [RENTANTA SEPTEMBRIE 2020]

Fie $f : B_2^3 \rightarrow B_2^2$, $f(x, y, z) = (f_1(x, y, z), f_2(x, y, z))$, unde:

- $f_1(x, y, z) = (x \oplus y)(y \oplus z)$
- $f_2(x, y, z) = 1$ d.d. secventa x, y, z este crescatoare (adica $x \leq y \leq z$).

Construiti tabelul de valori al lui f si scrieti f_1 si f_2 in FND si FNC.

index	x	y	z	$x \oplus y$	$y \oplus z$	$f_1(x, y, z)$	$f_2(x, y, z)$
(0)	0	0	0	0	0	0	1
(1)	0	0	1	0	1	0	1
(2)	0	1	0	1	1	1	0
(3)	0	1	1	1	0	0	1
(4)	1	0	0	1	0	0	0
(5)	1	0	1	1	1	1	0
(6)	1	1	0	0	1	0	0
(7)	1	1	1	0	0	0	1

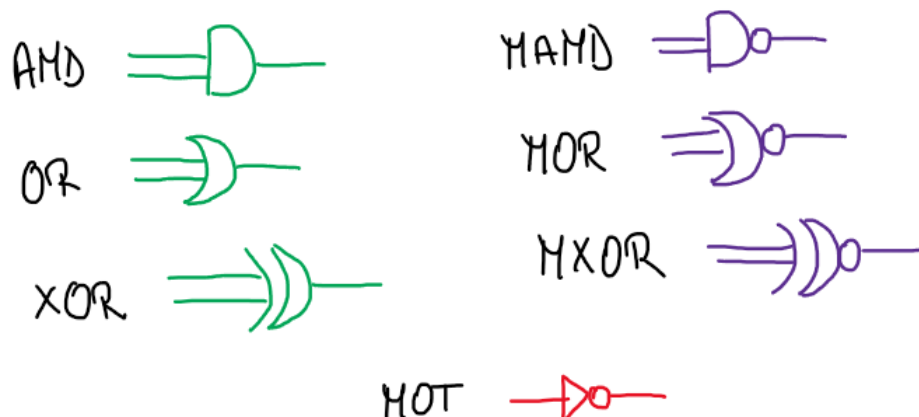
FND:

- $f_1 = \bar{x} \cdot y \cdot \bar{z} + x \cdot \bar{y} \cdot z$ (2)+(5)
- $f_2 = \bar{x} \cdot \bar{y} \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z + \bar{x} \cdot y \cdot z + x \cdot y \cdot z$ (0)+(1)+(3)+(7)

FNC:

- $f_1 = (x + y + z) \cdot (x + y + \bar{z}) \cdot (x + \bar{y} + \bar{z}) \cdot (\bar{x} + y + z) \cdot (x + y + \bar{z}) \cdot (x + y + z)$ (0) \cdot (1) \cdot (3) \cdot (4) \cdot (6) \cdot (7)
- $f_2 = (x + \bar{y} + z) \cdot (\bar{x} + y + z) \cdot (\bar{x} + y + \bar{z}) \cdot (\bar{x} + \bar{y} + z)$ (2) \cdot (4) \cdot (5) \cdot (6)

2 Porti logice



IMPORTANT!

Orice circuit se poate implementa folosind doar porti NOR, respectiv NAND. Cum? Stim ca orice circuit se poate implementa doar cu porti AND, OR si NOT => daca aratam ca putem sa implementam aceste 3 porti doar cu porti NOR, respectiv NAND, am demonstrat ca orice circuit se poate impementa doar cu porti NOR, respectiv NAND.

Pentru NOR:

- NEGATIA:

$$\bar{x} = x \text{ NOR } x$$

- AND:

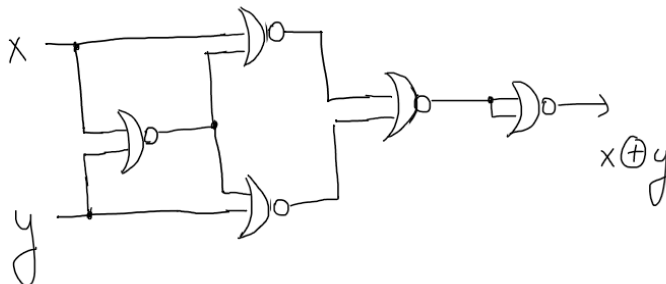
$$x \cdot y = \overline{\overline{x \cdot y}} = \overline{\bar{x} + \bar{y}} = \overline{\bar{x} \text{ NOR } \bar{y}} = (x \text{ NOR } x) \text{ NOR } (y \text{ NOR } y)$$

- OR:

$$x + y = \overline{\overline{x + y}} = \overline{\bar{x} \text{ NOR } \bar{y}} = (x \text{ NOR } x) \text{ NOR } (y \text{ NOR } y) \quad (1)$$

- XOR: Prin anumite "optimizari" in calcul putem obtine o formula cu doar 5 NOR-uri (vezi Exemplul 4 pentru demonstratie):

$$x \oplus y = [(x \text{ NOR } (x \text{ NOR } y)) \text{ NOR } (y \text{ NOR } (x \text{ NOR } y))] \text{ NOR } [(x \text{ NOR } (x \text{ NOR } y)) \text{ NOR } (y \text{ NOR } (x \text{ NOR } y))] \quad (2)$$



Pentru NAND:

- NEGATIA:

$$\bar{x} = \overline{x \cdot x} = x \text{ NAND } x$$

- AND:

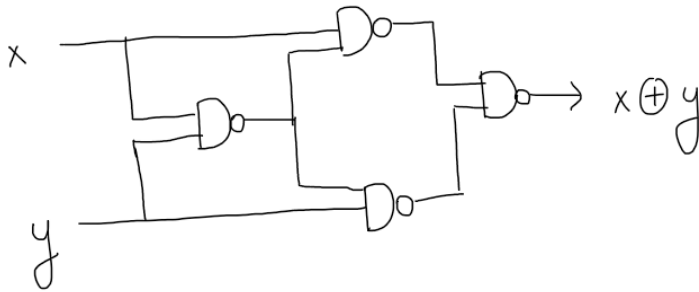
$$x \cdot y = \overline{\overline{x \cdot y}} = \overline{x \text{ NAND } y} = (x \text{ NAND } y) \text{ NAND } (x \text{ NAND } y)$$

- OR:

$$x + y = \overline{\overline{x + y}} = \overline{\bar{x} \cdot \bar{y}} = \bar{x} \text{ NAND } \bar{y} = (x \text{ NAND } x) \text{ NAND } (y \text{ NAND } y)$$

- XOR: Prin anumite "optimizari" in calcul, putem sa ajungem la o formula cu doar 4 NAND-uri (vezi Exemplul 5 pentru demonstratie):

$$x \oplus y = ((x \text{ NAND } y) \text{ NAND } x) \text{ NAND } ((x \text{ NAND } y) \text{ NAND } y)$$

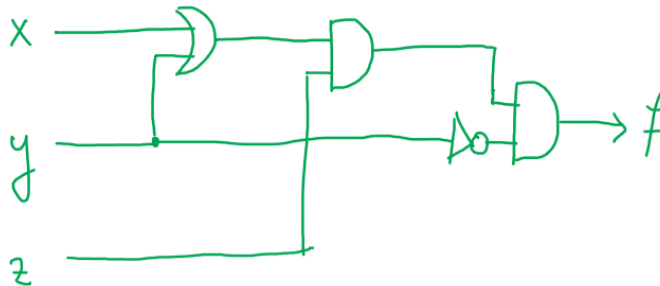


Ca sa intelegem mai bine cum se lucreaza cu aceste porti logice, vom face cateva exemple:

Exemplul 1 : Reprezentati prin porti logice functia:

$$f : \mathbb{B}_2^3 \rightarrow \mathbb{B}_2$$

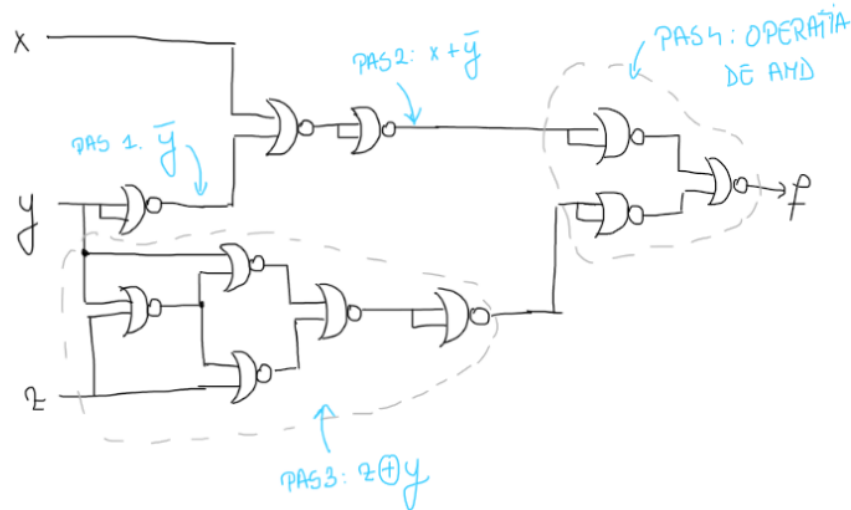
$$f(x, y, z) = (x + y) \cdot z \cdot \bar{y}$$



Exemplul 2 : Implementati functia $f : B_2^3 \rightarrow B_2^1$, $f(x) = (x + \bar{y}) \cdot (z \oplus y)$ doar cu porti NOR si apoi doar cu porti NAND.

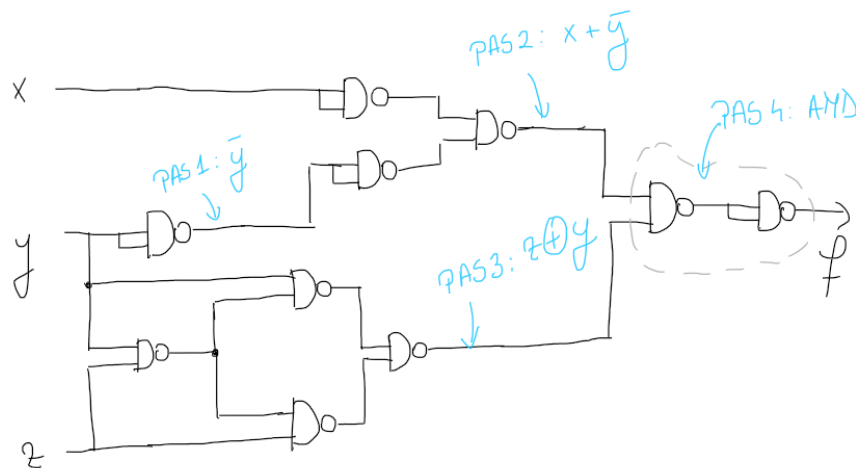
Varianta cu NOR:

$$f : B_2^3 \rightarrow B_2^1, f(x, y, z) = (x + \bar{y}) \cdot (z \oplus y)$$



Varianta cu NAND:

$$f : B_2^3 \rightarrow B_2^1, f(x, y, z) = (x + \bar{y}) \cdot (z \oplus y)$$



Exemplul 3 [RESTANTA SEPTEMBRIE 2020]

Implementati poarta NXOR printr-un circuit care contine doar porti NOR.

Prima metoda care ne vine in minte este sa expimam x NXOR y = $\bar{x} \cdot y + x \cdot \bar{y}$ si sa inlocuim operatiile de NEGATIE, AND si OR asa cum stim din formulele pentru NOR.

Aceasta rezolvare este corecta, dar rezulta in:

- 3 porti NOR de la NEGATIE

- 3+3 porti NOR de la AND
- 3 porti NOR de la OR

=> in total vom avea 12 porti NOR, ceea ce nu e ideal.

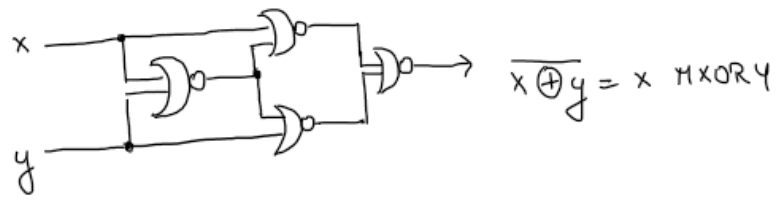
Dragulici nu specifica ca numarul de porti logice sa fie minim, dar cu siguranta nu se supara daca rezolvati cu numar minim. Si o sa va ajute si pe voi sa nu va incurcati, pentru ca a desena 12 porti doar pentru un XNOR nu e usor.

Asa ca vom incerca sa ajungem la numarul minim de porti cu care se poate reprezenta NXOR.

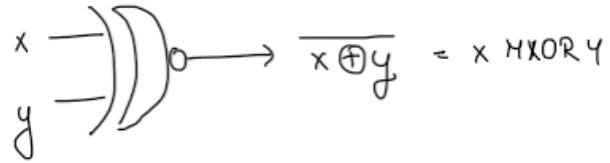
Keep in mind, la examen daca vreti sa folositi varianta "optimizata" cu 5 porti, trebuie sa aratati cum ati ajuns acolo.

$$\begin{aligned}
 x \text{ NXOR } y &= \overline{x \oplus y} \\
 x \text{ NOR } y &= \overline{x + y} \\
 \text{Stim ca:} \\
 x \oplus y &= x \cdot \bar{y} + y \cdot \bar{x} \\
 \Downarrow \\
 \overline{x \oplus y} &= \overline{x \cdot \bar{y} + y \cdot \bar{x}} \\
 \text{Stim ca } x \cdot \bar{x} &= 0 \text{ si } y \cdot \bar{y} = 0 \\
 \Rightarrow \text{pot sa adaug } x \cdot \bar{x} \text{ si } y \cdot \bar{y} & \\
 \overline{x \oplus y} &= \overline{x \cdot \bar{y} + y \cdot \bar{x} + x \cdot \bar{x} + y \cdot \bar{y}} = \\
 &= \overline{\underbrace{\bar{x}(x+y)}_A + \underbrace{\bar{y}(x+y)}_B} \\
 A &= \bar{x}(x+y) \\
 A &= \bar{A} = \overline{\bar{x} \cdot (x+y)} \stackrel{\text{de Morgan}}{=} \overline{\bar{x}} + \overline{(x+y)} = \\
 &= x + \overline{(x+y)} = x \text{ NOR } (x+y) = \\
 \Rightarrow A &= x \text{ NOR } (x \text{ NOR } y) \\
 \text{Analog } B &= y \text{ NOR } (x \text{ NOR } y) \\
 \overline{x \oplus y} &= \overline{[x \text{ NOR } (x \text{ NOR } y)] + [y \text{ NOR } (x \text{ NOR } y)]} = \\
 &= (x \text{ NOR } (x \text{ NOR } y)) \text{ NOR } (y \text{ NOR } (x \text{ NOR } y))
 \end{aligned}$$

Iar diagrama va fi:



$$\overline{x \oplus y} = x \text{ XOR } y$$



$$\overline{x \oplus y} = x \text{ XOR } y$$

Exemplul 4 Implementati poarta XOR printr-un circuit care contine doar porti NOR.

$$x \oplus y = \bar{x} \cdot y + x \cdot \bar{y}$$

$$\text{Stim ca } x \cdot \bar{x} = 0$$

$$y \cdot \bar{y} = 0$$

$$2) \quad x \oplus y = \bar{x} \cdot y + x \cdot \bar{y} + x \cdot \bar{x} + y \cdot \bar{y} =$$

$$= \bar{x} \cdot (x + y) + \bar{y} \cdot (x + y) =$$

$$\stackrel{A=\bar{A}}{\rightarrow} = \overline{\bar{x} \cdot (x + y)} + \overline{\bar{y} \cdot (x + y)} =$$

$$\stackrel{\text{de Morgan}}{\rightarrow} = \overline{(\bar{x} + \overline{(x + y)})} + \overline{(\bar{y} + \overline{(x + y)})} =$$

$$\stackrel{A=\bar{A}}{\rightarrow} = \underbrace{\overline{x + \overline{(x + y)}}}_B + \underbrace{\overline{y + \overline{(x + y)}}}_C$$

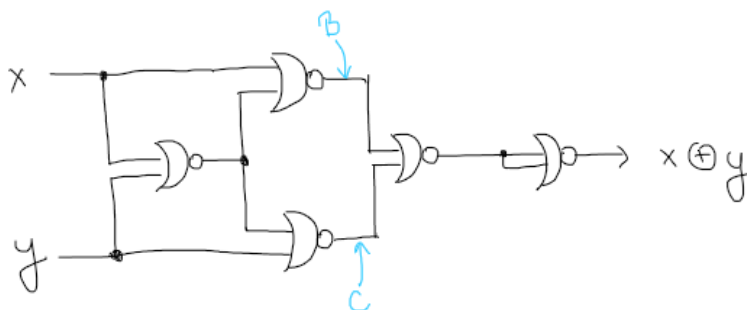
$$x \oplus y = B + C \stackrel{A=\bar{A}}{\rightarrow} \overline{\overline{B + C}} = \overline{B \text{ NOR } C} =$$

$$= (B \text{ NOR } C) \text{ NOR } (B \text{ NOR } C)$$

$$\text{unde } B = \overline{x + \overline{(x + y)}} = x \text{ NOR } (x \text{ NOR } y)$$

$$C = \overline{y + \overline{(x + y)}} = y \text{ NOR } (x \text{ NOR } y)$$

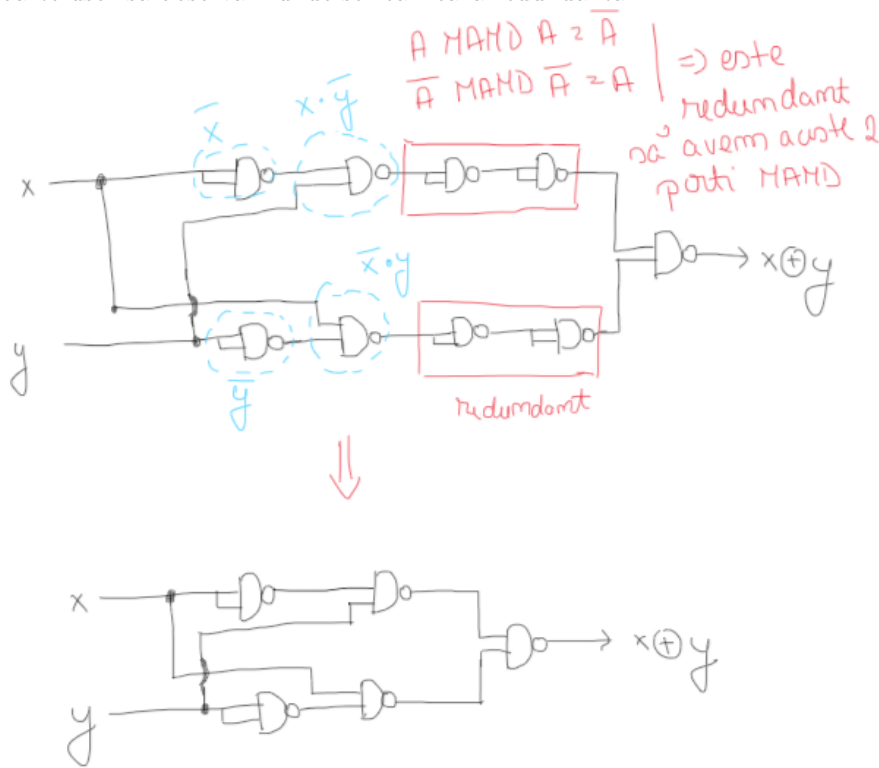
Iar diagrama va fi:



Exemplul 5 Implementati poarta NXOR printr-un circuit cu numar minim de porti si doar cu porti NAND.

$$\begin{aligned}
 x \oplus y &= x \cdot \bar{y} + \bar{x} \cdot y = \overline{\overline{x \cdot \bar{y}} + \overline{\bar{x} \cdot y}} = \\
 &= \overline{(\overline{x \cdot \bar{y}}) \cdot (\overline{\bar{x} \cdot y})} = \\
 &= \overline{x \cdot (y \text{ NAND } y) \cdot ((x \text{ NAND } x) \cdot y)} = \\
 &= \overline{(x \text{ NAND } (y \text{ NAND } y)) \text{ NAND } (x \text{ NAND } (y \text{ NAND } y))} \cdot \\
 &\quad \cdot ((x \text{ NAND } x) \text{ NAND } y) \text{ NAND } ((x \text{ NAND } x) \text{ NAND } y)
 \end{aligned}$$

Chiar daca aceasta nu este forma cu numar minim de porti logice, vom desena diagrama pentru ca asa ne va fi foarte usor sa observam unde se realizeaza redundanta.

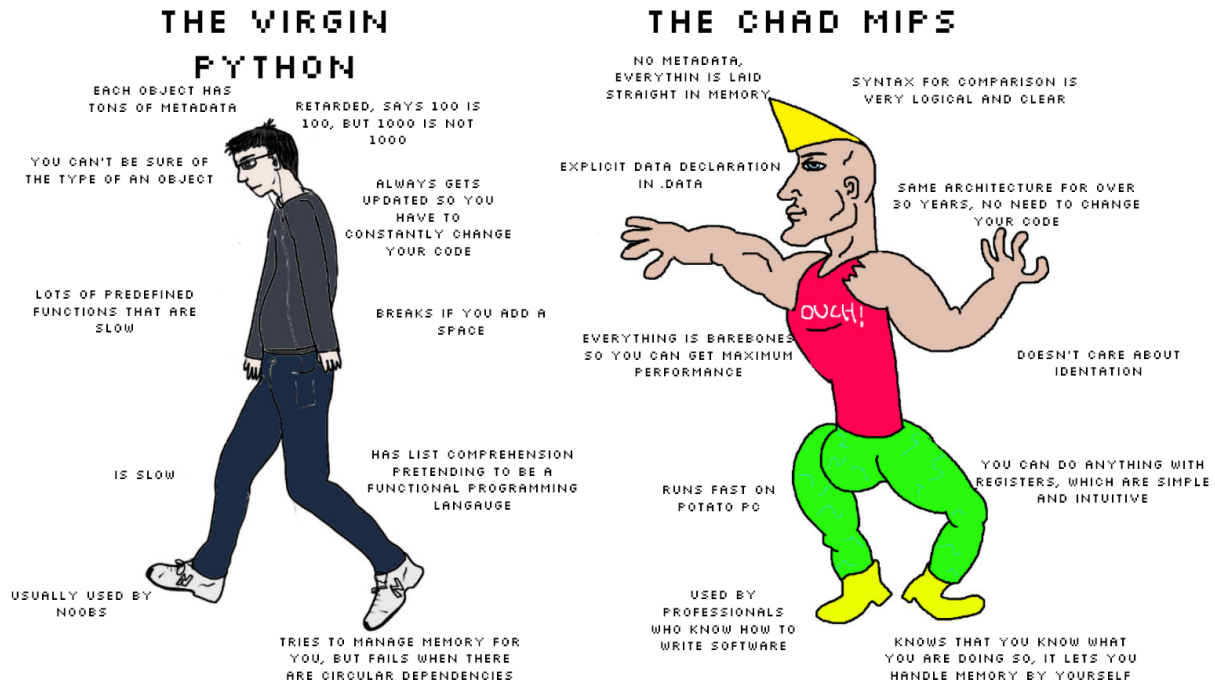


$$\Rightarrow x \oplus y = ((x \text{ NAND } x) \text{ NAND } y) \text{ NAND } ((y \text{ NAND } y) \text{ NAND } x)$$

Exemplul 6 Implementati poarta NXOR printr-un circuit cu numar minim de porti si doar cu porti NAND.

Avand atatea exemple mai sus, incercati exercitiul acesta singuri.

3 MIPS



3.1 Șiruri de caractere

Sunt tablouri unidimensionale de un byte și sunt finalizate cu caracterul `'\0'`. Această convenție de a termina șirurile de caractere cu `'\0'` este comună în multe limbaje de programare. În felul acesta se pot parcurge caracterele, iar când se ajunge la capătul șirului să se oprească, fără a mai fi nevoie de o variabilă auxiliară în care să se țină lungimea șirului.

Pentru a verifica dacă un caracter este egal cu `'\0'` se poate folosi `beqz`.

Exemple:

Problema 1 Să se afișeze pe ecran lungimea unui șir de caractere.

```
1 .data
2     str: .asciiz "Sir de caractere" # sirul de caractere
3 .text
4 main:
5
6     li $t0, 0                       # counterul
7     lb $t1, str($t0)                # iau primul caracter din sir
8
9     loop:
10
11         beqz $t1, exit              # daca am ajuns la sfarsitul sirului ne oprim
12
13         addi $t0, 1
14         lb $t1, str($t0)            # ia urmatorul caracter din sir
15         j loop
16
17 exit:
18
19     move $a0, $t0
20     li $v0, 1
21     syscall
22
23     li $v0, 10
24     syscall
```

PRINT BYTE

Dacă avem un caracter stocat într-un registru îl putem afișa pe ecran. Apelul de sistem pentru asta are codul 11. În \$a0 vom copia caracterul pe care vrem să-l afișăm.

Exemplu PRINT BYTE:

```

1 .data
2     ch:.byte 'a'
3 .text
4 main:
5
6     lb $a0, ch # copiem in $a0 caracterul pe care vrem sa-l afisam
7     li $v0, 11 # punem in $v0 codul pentru apelul de sistem, adica 11
8     syscall    # afiseaza caracterul pe ecran
9
10    li $v0, 10
11    syscall

```

printbyte.s

READ STRING

Avem apel de sistem pentru citirea șirurilor de caractere de la tastatură. Codul pentru acest apel de sistem este 8.

Pentru a citi un șir de caractere de la tastatură trebuie să facem următoarele lucruri:

- Punem în registrul \$a0 adresa de memorie la care vrem să reținem șirul de caractere
- Punem în registrul \$a1 dimensiunea maximă a șirului.
- Punem în registrul \$v0 codul pentru apelul de sistem, adică 8.

Exemplu READ STRING:

```

1 .data
2     str:.space 100 # sir de caractere de lungime 99
3                     # lungimea este 99, nu 100
4                     # deoarece ultimul caracter
5                     # este '\0'
6 .text
7 main:
8
9     # citeste sirul de caractere
10    la $a0, str      # pune in $a0 adresa de memorie la
11                     # care vreau sa retin sirul de caractere
12    li $a1, 99        # dimensiunea maxim a sirului
13    li $v0, 8         # codul pentru apelul de sistem este 8
14    syscall
15
16    # afiseaza sirul de caractere
17    la $a0, str
18    li $v0, 4
19    syscall
20
21    # exit
22    li $v0, 10
23    syscall

```

readstring.s

Mai multe exerciții

Problema 1: Se citește un șir de caractere de dimensiune maximă 99. Să se afișeze pe ecran caracterele de pe poziții pare.

Problema 2: Se dă un șir de caractere la nivel de memorie, să se modifice șirul adăugând un $+ 1$ pe codul ASCII al fiecărui element.

Exemplu: "abc xyz" $\rightarrow$ "bcd!yz{"

Problema 3: Prima parte din problema din prima zi de advent of code din 2017:

<https://adventofcode.com/2017/day/1>

3.2 Lucrul cu numere în virgulă mobilă



Regiștri

În MIPS avem regiștri $\$f0$ - $\$f31$ dedicați pentru lucrul cu numere în virgulă mobilă. Fiecare registru este pe 32 de biți, deci poate ține un single. Pentru double, sunt folosiți doi regiștri (unul după altul). De exemplu dacă am vrea să ținem un double în regiștrii $\$f0$, respectiv $\$f1$, 32 de biți vor fi ținuți în $\$f0$, iar restul vor fi ținuți în $\$f1$.

Tipuri de date

Se declară ca și celelalte tipuri de date în `.data`.

Pentru numerele în virgulă mobilă avem tipurile:

- `.float` - tip de date pe 32 de biți folosit pentru a stoca numere single
- `.double` - tip de date pe 64 de biți folosit pentru a stoca numere double

Exemple de date declarate

a:.float 10.54
b:.double 20.31

Instrucțiuni pentru transferarea datelor din regiștri și invers

Când lucrăm cu single și double, de regulă postfixăm numele instrucțiunii noastre cu `.s`, respectiv `.d`. Din cauza faptului că pentru double sunt folosiți doi regiștri, instrucțiunile pentru double funcționează doar pe regiștrii cu număr par (ex: $\$f0$, $\$f2$, $\$f4$, etc). În unele implementări de MIPS, regula rămâne valabilă și pentru instrucțiunile pentru numere single.

Instrucțiuni pentru încărcarea valorilor în regiștri:

`l.s/ l.d  $\$fd$ , mem` (load single/ load double, încarcă în registrul $\$fd$, valoarea aflată în memorie în `mem`. În cazul la double, va fi folosit și următorul registru pentru a se stoca în el. Este valabil pentru orice instrucțiune de transfer al datelor pe double)

`li.s/ li.d  $\$fd$ , const` (Încarcă o valoarea constantă `const` în registrul $\$fd$)

Instrucțiuni pentru salvarea valorilor din regiștri în memorie:

`s.s/ s.d  $\$fs$ , mem` (store single/ store double, salvează valoarea din registrul $\$fs$ la adresa `mem`)

Exemple de folosire a acestor instrucțiuni:

`l.s  $\$f0$ , a #` încarcă în registrul $\$f0$ valoarea din `a` (adică 10.54)
`l.d  $\$f2$ , b #` încarcă în regiștrii $\$f2$ și $\$f3$ valoarea din `b` (adică 20.31)
`li.s  $\$f4$ , 15.3 #` încarcă în registrul $\$f4$ valoarea constantă 15.3
`li.d  $\$f6$ , 21.9 #` încarcă în registrul $\$f6$ valoarea constantă 21.9
`s.s  $\$f4$ , a #` salvează valoarea din registrul $\$f4$ la adresa de memorie a lui `a`

s.d \$f6, b # salvează valoarea din registrul \$f6 (respectiv \$f7) la adresa de memorie a lui b.

instrucțiuni aritmetice

add.s/ add.d *dest*, *src1*, *src2* (adună numerele din regiștri *src1* și *src2* și salvează rezultatul în registrul *dest*. Și la aceste instrucțiuni rămâne valabil comportamentul pentru duble, sunt folosiți câte doi regiștri)

sub.s/ sub.d *dest*, *src1*, *src2* (asemănător cu add.s/ add.d, realizează scăderea dintre valorile din registrul *src1* și *src2*, iar rezultatul îl salvează în registrul *dest*. sub.d este pentru double)

mul.s/ mul.d *dest*, *src1*, *src2* (realizează operația $dest = src1 \cdot src2$)

div.s/ div.d *dest*, *src1*, *src2* (realizează operația $dest = src1 / src2$)

neg.s/ neg.d *dest*, *src* (realizează operația $dest = -src$)

abs.s/ abs.d *dest*, *src* (realizează operația $dest = ||src||$)

IO + apeluri de sistem

Coduri pentru apeluri de sistem:

2 - PRINT SINGLE (afișează pe ecran un număr single, se încarcă în \$f12 valoarea de afișat, în v0 valoarea 2 (codul pentru apelul de sistem), iar apoi se scrie syscall)

3 - PRINT DOUBLE (afișează pe ecran un număr double, se încarcă în \$f12, respectiv \$f13 valoarea de afișat, în v0 valoarea 3 (codul pentru apelul de sistem), iar apoi se scrie syscall)

6 - READ SINGLE (citește un single de la tastatură, se încarcă în v0 valoarea 6 (codul pentru apelul de sistem), apoi se scrie syscall. După ce a reușit să citească numărul de la tastatură, sistemul ne va returna valoarea citită în registrul \$f0)

7 - READ DOUBLE (citește un double de la tastatură, se încarcă în v0 valoarea 7 (codul pentru apelul de sistem), apoi se scrie syscall. După ce a reușit să citească numărul de la tastatură, sistemul ne va returna valoarea citită în regiștri \$f0, respectiv \$f1)

Exemple de apeluri de sistem

READ SINGLE

li \$v0, 6

syscall mov.s \$f2, \$f0 # În \$f0 am primit valoarea citită de la tastatură. O vom copia în registrul \$f2 cu ajutorul lui mov.s (folosim mov.d în cazul în care avem double). Sintaxa pentru mov.s este mov.s *regd*, *regs*. Instrucțiunea copiază valoarea din registrul *regs* în

registru *regd* (analog pentru *mov.d*).

Exerciții și probleme

Problema 1: Se dau două numere de tip *single* stocate în memorie, să se interschimbe valorile celor două numere și să se afișeze pe ecran noile valori.

```
1 .data
2
3     x:.float 11.4    # x este un single declarat in memorie cu valoarea implicita 11.4
4     y:.float 34.5    # y -> single cu valoarea 34.5
5
6 .text
7 main:
8
9     l.s $f0, x        # incarc in $f0 valoarea din x
10    l.s $f2, y        # incarc in $f2 valoarea din y
11                      # am ales $f2 in loc de $f1
12                      # deoarece in unele medii de lucru
13                      # nu putem folosi $f-urile impare
14                      # nici pentru operatii cu single
15
16    s.s $f0, y        # pun in y valoarea din f0 (adica ce era inainte in x)
17    s.s $f2, x        # pun in x valoarea din f2 (adica ce era inainte in y)
18
19    li $v0, 2          # codul pentru a afisa un single pe ecran
20    mov.s $f12, $f2    # pun in $v12 valoarea din f2 (adica ce avem acum in x)
21    syscall           # afisez pe ecran
22
23    li $v0, 11         # afisez spatiu intre rezultate
24    li $a0, ' '        # pun in a0 caracter pentru spatiu
25                      # in unele medii (spim) putem face direct
26                      # fara a mai declara in memorie spatiul
27    syscall
28
29    li $v0, 2
30    mov.s $f12, $f0    # acum afisez valoarea din $f0 (adica ce avem acum in y)
31    syscall
32
33    li $v0, 10
34    syscall
```

problema1_1.s

Branch instructions

Sunt asemănătoare cu cele de la *word*-uri, deși diferă puțin sintaxa și modul de lucru. Fiecare instrucțiune de verificare a condițiilor începe cu "c.", este urmată de "eq", "lt" sau "le", iar apoi este urmată de ".s" sau ".d" (în funcție dacă folosim *single* sau *double*).

Instrucțiunile:

c.eq.s/ c.eq.d fs, ft (comparison equal, verifică dacă fs și ft sunt egale (deși nu aș recomanda să faceți asta vreodată cu *float*-uri... în niciun limbaj de programare). Dacă sunt egale, atunci setează bitului de condiție (parte din procesor) valoarea 1, altfel valoarea 0)

c.lt.s/ c.lt.d fs, ft (comparison less than, verifică dacă fs este mai mic decât ft, iar dacă este, setează valoarea bitului de condiție valoarea 1, altfel valoarea 0)

c.le.s/ c.le.d fs, ft (comparison less than or equal, verifică dacă fs este mai mic sau egal cu

ft, iar dacă este, setează valoarea bitului de condiție valoarea 1, altfel valoarea 0)

Asta nu e tot ce trebuie să facem, aceste instrucțiuni doar au modificat valoarea bitului de condiție. noi acum trebuie ne ducem la un branch dacă această valoare este 1 sau 0.

Pentru asta avem următoarele instrucțiuni:

bc1t label (continuă executarea de la linia unde este *label*, dacă bitul de condiție are valoarea 1)

bc1f label (continuă executarea de la linia unde este *label*, dacă bitul de condiție are valoarea 0)

Exerciții și probleme

Problema 1: Să se afișeze pe ecran toate valorile pozitive mai mici sau egale cu n (single citit de la tastatură) pornind de la 0 și incrementând cu un step de 0.1.

```
1 .data
2
3     n:.float 0.0                # numarul n, il vom tine si in memorie (doar in scop didactic)
4
5 .text
6 main:
7
8     li $v0, 6                    # citeste n
9     syscall
10    s.s $f0, n                  # vom pastra in $f0 valoarea lui n
11
12    li.s $f2, 0.1                # pasul cu care vom incrementa counterul
13    li.s $f4, 0.0                # counterul nostru, initializat cu 0
14
15    loop:
16
17        c.le.s $f0, $f4          # daca counterul nostru este >= f0, atunci am terminat
18        # deci seteaza bitului de conditie valoarea 1
19        bc1t exit                # daca bitul de conditie este 1, am terminat
20        # din pacate nu avem branching ca la word-uri
21        # si nici nu putem verifica daca un numar e mai mare(sau egal)
22        # cu altul, trebuie sa verificam daca sunt mai mici(sau egale)
23
24        li $v0, 2                # afiseaza numarul curent
25        mov.s $f12, $f4
26        syscall
27
28        li $v0, 11
29        li $a0, ' ',
30        syscall
31
32        add.s $f4, $f4, $f2      # incrementeaza counterul cu step
33        # adica $t4 = $t4 + $t2
34
35        j loop
36
37    exit:
38
39        li $v0, 10
40        syscall
```

Conversii

Putem copia bit cu bit valorile din regiștri normali în regiștri în virgulă mobilă (și invers) folosind următoarele instrucțiuni:

`mtc1 rs, fd` (copiază din registrul normal *rs*, în registrul pentru floating point *fd*)

`mfc1 rd, fs` (copiază din registrul pentru floating point *fs*, în registrul normal *rd*)

Aceste instrucțiuni nu fac conversie între tipuri, ci doar copiază bit cu bit valorile. Pentru a converti avem instrucțiunea `cvt`, care se postfixează cu `".dest.sursa"`, unde *dest* și *sursa* reprezintă tipul către care vrem să convertim, respectiv tipul de la care convertim. Dest și sursă pot avea valorile "s", "d", respectiv "w" (pentru word).

Sintaxa: `cvt.dest.sursa ds, sr ->` convertește valoarea din *sr* (de tipul *sursa*) în valoare de tip *dest* și salvează rezultatul în registrul *ds*.

Exerciții și probleme

Problema 1: Se citește un număr de tip single de la tastatură, să se convertească la word, iar apoi să se afișeze pe ecran rezultatul.

```

1  .data
2  .text
3  main:
4
5      li $v0, 6
6      syscall
7
8      cvt.w.s $f2, $f0 # convertim pe $f0 (citit) in word, si tinem rezultatul in $f2
9                        # noi am vrea sa-l tinem in $t0 pentru ca este word, dar compilerul nu
10                       # ne lasa
11      mfc1 $t0, $f2    # asa ca trebuie sa punem un registru pentru float-uri ca destinatie
12                       # punem rezultatul din $f2 in $t0 (in $t0, este reprezentat intern ca
13                       # word, dar este in registru de float, deci il copiem intr-un registru
14                       # pentru word-uri)
15      li $v0, 1        # afisam pe ecran rezultatul
16      move $a0, $t0
17      syscall
18
19      li $v0, 10
20      syscall

```

Mai multe exerciții

Problema 1: Se citesc $n \in \mathbb{N}^*$ și n numere de tip single. Să se calculeze media lor aritmetică.

References

- [1] Dumitru Daniel Drăgulici. *Curs Arhitectura Sistemelor de Calcul*.
- [2] Larisa Dumitrache. *Tutoriat 2019*
- [3] Bogdan Macovei. *Laboratoare ASC 2019/ 2020*
- [4] Advent Of Code. *Day 1 2017*